ARTIFICIAL INTELLIGENCE

Name: _____ **Stream:** **BSc. Computer Science (Sem-V)** **Roll No.:** ____

INDEX

Sr.	Practical Name	Date	Signature
1	Breadth First Search (BFS) & Iterative Depth First Search (DFS)		
2	A* Search and Recursive Best-First Search (RBFS)		
3	Decision Tree Learning		
4	Feedforward Backpropagation Neural Network		
5	Support Vector Machine (SVM)		
6	Naïve Bayes Classifier		
7	K-Nearest Neighbors (K-NN)		
8	Demo of TensorFlow Tools		
9	Adaboost Learning Algorithm		
10	Association Rule Mining		

1 PRACTICAL - 1: Breadth First Search & Iterative Depth First Search

A. Implement the Breadth First Search algorithm to solve a given problem.

Theory:

Breadth First Search (BFS) is an algorithm used to traverse or search a graph or tree data structure. It explores all the vertices of a graph in breadth-first order, starting from a specified vertex known as the “source” vertex. BFS is often used to find the shortest path between two vertices in an unweighted graph or to visit all the vertices in a connected component of a graph.

The BFS algorithm works as follows:

- **a)** Start by initializing a queue and a set to keep track of visited vertices.
- **b)** En-queue the source vertex into the queue and mark it as visited.
- **c)** Repeat the following steps until the queue becomes empty:
- **d)** De-queue a vertex from the front of the queue.
- **e)** Process the vertex (print it, store it, or perform any other desired operation).
- **f)** En-queue all the unvisited neighbors of the vertex into the queue and mark them as visited.
- **g)** The algorithm terminates when the queue becomes empty, indicating that all reachable vertices have been processed.

Code with OUTPUT:

```
graph = {
    '5': ['3', '7'],
    '3': ['2', '4'],
    '7': ['8'],
    '2': [],
    '4': ['8'],
    '8': []
}

visited = []  # List for visited nodes.
queue = []    # Initialize a queue

def bfs(visited, graph, node):  # Function for BFS
    visited.append(node)
    queue.append(node)

    while queue:  # Creating loop to visit each node // FIFO
        m = queue.pop(0)
        print(m, end=" ")

        for neighbour in graph[m]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

# Driver Code
print("Following is the Breadth-First Search")
bfs(visited, graph, '5')  # Function call
```

Output:

```
Following is the Breadth-First Search
5 3 7 2 4 8
```

B. Implement the Iterative Depth First Search algorithm to solve the same problem.

Theory:

Iterative Depth First Search (DFS) is an algorithm used to traverse or search a graph or tree data structure. It explores all the vertices of a graph in depth-first order, starting from a specified vertex known as the “source” vertex.

The iterative DFS algorithm works as follows:

- **a)** Start by initializing a stack and a set to keep track of visited vertices.
- **b)** Push the source vertex onto the stack and mark it as visited.
- **c)** Repeat the following steps until the stack becomes empty:
- **d)** Pop a vertex from the top of the stack.
- **e)** Process the vertex (print it, store it, or perform any other desired operation).
- **f)** Retrieve all the unvisited neighbors of the vertex.
- **g)** For each unvisited neighbor, push it onto the stack and mark it as visited.
- **h)** The algorithm terminates when the stack becomes empty, indicating the vertices have been processed.

Code with OUTPUT:

```
graph = {
    '5': ['3', '7'],
    '3': ['2', '4'],
    '7': ['8'],
    '2': [],
    '4': ['8'],
    '8': []
}

visited = []  # List for visited nodes.
stack = []    # Initialize a stack

def dfs(visited, graph, node):
    stack.append(node)

    while stack:
        m = stack.pop()
        if m not in visited:
            print(m, end=" ")
            visited.append(m)
            # Push neighbors in reverse order to ensure left node is processed first
            for neighbour in reversed(graph[m]):
                if neighbour not in visited:
                    stack.append(neighbour)

# Driver Code
print("Following is the Depth-First Search")
dfs(visited, graph, '5')
```

Output:

```
Following is the Depth-First Search
5 3 2 4 8 7
```

C. Compare the performance and efficiency of both algorithms.

Solution:

Criteria	BFS	Iterative DFS
Time Complexity	$O(V + E)$	$O(V + E)$
Space Complexity	$O(V)$	$O(d)$
Completeness	Yes (for finite graphs)	No (for infinite graphs)
Optimality	Yes (for unweighted graphs)	No
Traversal Order	Level by level	Deepest nodes first
Best Use Cases	Shortest path, broad-level search	Deep solutions, exhaustive search

2 PRACTICAL - 2: A* Search and Recursive Best-First Search

Aim:

1. Implement the A* Search algorithm for solving a path finding problem.
2. Implement the Recursive Best-First Search algorithm for the same problem.
3. Compare the performance and effectiveness of both algorithms.

Part 1: A* Search algorithm

Theory:

A* (A-star) is a widely used search algorithm that combines the best features of both Dijkstra's algorithm and heuristic search. It is commonly applied to solve the path-finding problem in graphs or grids, where the goal is to find the shortest path from a start node to a goal node.

The A* algorithm works by maintaining two main values for each node: the cost to reach the node from the start node (known as g-value), and an estimate of the cost from the node to the goal node (known as h-value). It uses a priority queue, typically implemented as a min-heap, to prioritize the nodes for exploration based on their f-value, which is the sum of the g-value and h-value.

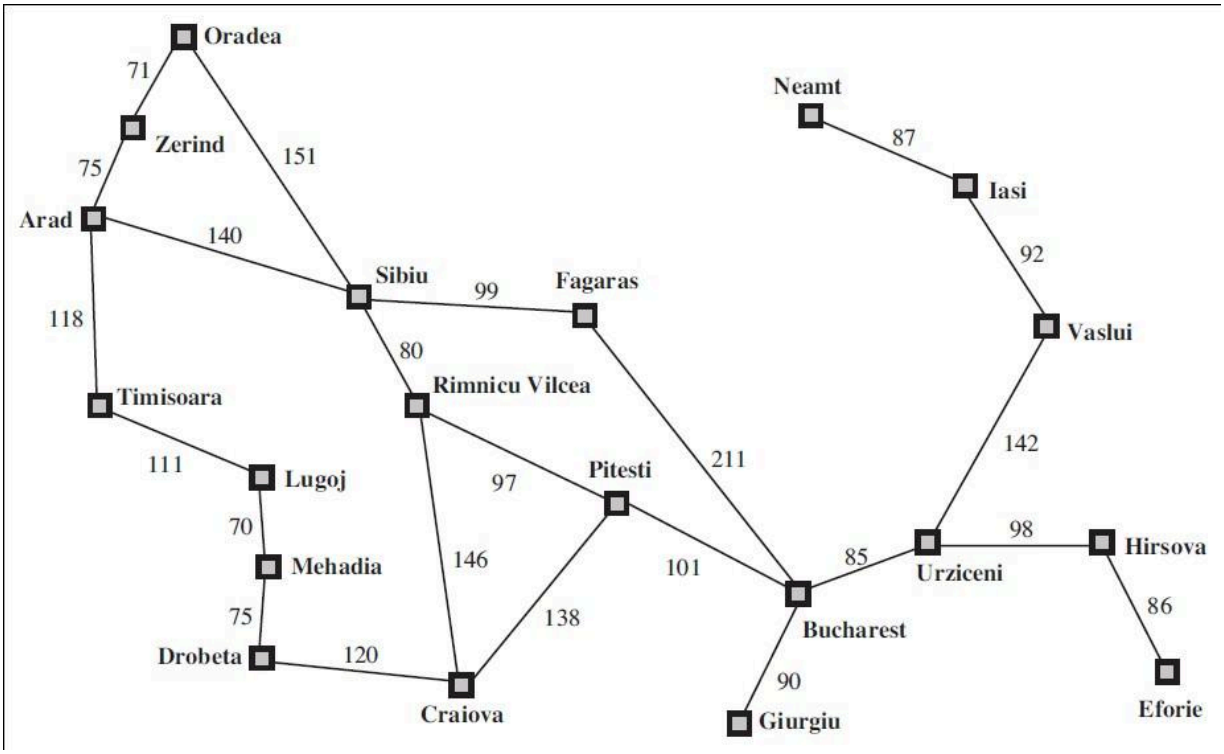
The A* algorithm follows these steps:

- **a)** Initialize the open list, closed list, and set the g-value of the start node to 0.
- **b)** Calculate the h-value for each node in the graph or grid based on a heuristic function. The heuristic function estimates the cost from each node to the goal node. Common heuristic functions include Euclidean distance, Manhattan distance, or any other admissible and consistent heuristic.
- **c)** Enqueue the start node to the open list with its f-value as the priority.
- **d)** Repeat the following steps until the open list becomes empty or the goal node is reached:
  - **I.** Dequeue the node with the lowest f-value from the open list. This node becomes the current node.
  - **II.** If the current node is the goal node, the algorithm terminates, and the path has been found.
  - **III.** Add the current node to the closed list to mark it as visited.
  - **IV.** Explore the neighboring nodes of the current node:
    - **i.** Calculate the tentative g-value for each neighbor by adding the cost to reach the neighbor from the current node to the g-value of the current node.
    - **ii.** If the neighbor is not in the closed list or its tentative g-value is lower than its current g-value:
      - **1)** Update the g-value of the neighbor to the new lower value.
      - **2)** Calculate the f-value of the neighbor by adding its g-value and h-value.
      - **3)** If the neighbor is not in the open list, enqueue it with its f-value as the priority.
      - **4)** If the neighbor is already in the open list, update its priority if the new f-value is lower.
      - **5)** Set the parent of the neighbor to the current node.
- **e)** If the open list becomes empty before reaching the goal node, there is no path available.
- **f)** Once the goal node is reached, reconstruct the path by following the parent pointers from the goal node to the start node.

The A* algorithm is both complete (able to find a solution if one exists) and optimal (guaranteed to find the shortest path) under certain conditions. The heuristic used must be admissible, meaning it never overestimates the actual cost to reach the goal node. Additionally, the heuristic must be consistent (or monotonic), meaning the estimated cost from a node to its successor plus the heuristic value of the successor is always less than or equal to the estimated cost from the current node to the goal node.

A* is widely used in various applications such as path planning, robotics, navigation systems, and game AI due to its efficiency and optimality. It efficiently explores the search space by prioritizing nodes that are most likely to lead to the goal while considering the actual cost from the start node.

We consider the Map of Romania problem and solve it using A* Search.



Python code:

```
import heapq

romania_map = {
    'Arad': {'Zerind': 75, 'Timisoara': 118, 'Sibiu': 140},
    'Zerind': {'Arad': 75, 'Oradea': 71},
    'Timisoara': {'Arad': 118, 'Lugoj': 111},
    'Sibiu': {'Arad': 140, 'Oradea': 151, 'Fagaras': 99, 'Rimnicu Vilcea': 80},
    'Oradea': {'Zerind': 71, 'Sibiu': 151},
    'Lugoj': {'Timisoara': 111, 'Mehadia': 70},
```

```
    'Fagaras': {'Sibiu': 99, 'Bucharest': 211},
    'Rimnicu Vilcea': {'Sibiu': 80, 'Pitesti': 97, 'Craiova': 146},
    'Mehadia': {'Lugoj': 70, 'Drobeta': 75},
    'Drobeta': {'Mehadia': 75, 'Craiova': 120},
    'Craiova': {'Drobeta': 120, 'Rimnicu Vilcea': 146, 'Pitesti': 138},
    'Pitesti': {'Rimnicu Vilcea': 97, 'Craiova': 138, 'Bucharest': 101},
    'Bucharest': {'Fagaras': 211, 'Pitesti': 101, 'Giurgiu': 90, 'Urziceni': 85},
    'Giurgiu': {'Bucharest': 90},
    'Urziceni': {'Bucharest': 85, 'Hirsova': 98, 'Vaslui': 142},
    'Hirsova': {'Urziceni': 98, 'Eforie': 86},
    'Eforie': {'Hirsova': 86},
    'Vaslui': {'Urziceni': 142, 'Iasi': 92},
    'Iasi': {'Vaslui': 92, 'Neamt': 87},
    'Neamt': {'Iasi': 87}
}

class Node:
    def __init__(self, city, cost, parent=None):
        self.city = city
        self.cost = cost
        self.parent = parent

    def __lt__(self, other):
        return self.cost < other.cost

def astar_search(graph, start_city, goal_city):
    open_list = []
    closed_set = set()

    start_node = Node(start_city, cost=0)
    heapq.heappush(open_list, start_node)

    while open_list:
        current_node = heapq.heappop(open_list)

        if current_node.city == goal_city:
            path = []
            while current_node:
                path.append(current_node.city)
                current_node = current_node.parent
            return path[::-1]

        closed_set.add(current_node.city)

        for neighbor, distance in graph[current_node.city].items():
            if neighbor not in closed_set:
                new_cost = current_node.cost + distance
                new_node = Node(neighbor, new_cost, current_node)
                heapq.heappush(open_list, new_node)

    return None

start_city = 'Arad'
goal_city = 'Bucharest'
path = astar_search(romania_map, start_city, goal_city)

if path:
    print("Path found:", path)
else:
    print("No path found")
```

Output:

```
Path found: ['Arad', 'Sibiu', 'Rimnicu Vilcea', 'Pitesti', 'Bucharest']
```

Part 2: Recursive Best-First Search algorithm

Theory:

Recursive Best-First Search (RBFS) is an informed, memory-bounded search algorithm derived from A* that avoids storing the entire search tree explicitly. It uses f-values (g-cost plus heuristic h-cost) to prioritize node exploration.

The algorithm operates recursively by maintaining a limit and updating alternative values for successors during backtracking, allowing it to reconsider previously rejected paths if necessary. RBFS terminates upon finding a path or exhausting options, making it memory-efficient for large state spaces.

Python code for Recursive Best-First Search algorithm

```
from queue import PriorityQueue

class Node:
    def __init__(self, state, parent=None, f=float('inf')):
        self.state = state
        self.parent = parent
        self.f = f

def rbfs(start, goal):
    f_limit = float('inf')
```

```
    stack = [(Node(start, f=0), f_limit)]
    visited = set()

    while stack:
        (node, f) = stack.pop()
        visited.add(node.state)

        if node.state == goal:
            path = []
            cost = node.f
            while node is not None:
                path.append(node.state)
                node = node.parent
            return list(reversed(path)), cost

        successors = []
        for neighbor, cost in get_neighbors(node.state):
            if neighbor not in visited:
                child = Node(neighbor, parent=node)
                child.f = max(child.parent.f, cost)
                successors.append(child)

        if len(successors) == 0:
            continue

        successors.sort(key=lambda x: x.f)
        best = successors[0]

        if best.f > f_limit:
            return None, best.f

        alternative = successors[1].f if len(successors) > 1 else float('inf')
        stack.append((best, min(f_limit, alternative)))

    return None, float('inf')

def get_neighbors(state):
    # Define the successors for each state with their associated costs (simplified example).
    successors = {
        1: [(2, 3), (3, 5)],
        2: [(1, 3), (4, 7)],
        3: [(1, 5), (5, 2)],
        4: [(2, 7), (6, 4)],
        5: [(3, 2), (7, 6)],
        6: [(4, 4), (8, 8)],
        7: [(5, 6), (8, 5)],
        8: [(6, 8), (7, 5)],
    }
    return successors.get(state, [])

if __name__ == '__main__':
    start_state = 1
    goal_state = 8
    path, cost = rbfs(start_state, goal_state)

    if path not in [None, False]:
        print(f"Optimal path from {start_state} to {goal_state}:")
        print(" -> ".join(map(str, path)))
        print(f"Total cost: {cost}")
    else:
        print("No path found.")
```

Output:

```
Optimal path from 1 to 8:
1 -> 2 -> 4 -> 6 -> 8
Total cost: 8
```

Part 3: Comparing A* and RBFS

The comparison between A* and RBFS involves evaluating time complexity, space complexity, and overall behavior:

- **Efficiency:**
  - **Time Complexity:** A* generally runs faster or explores fewer nodes with an admissible heuristic, whereas RBFS can suffer from redundant exploration and backtracking, leading to exponential time complexity in worse cases.
  - **Space Complexity:** RBFS excels in memory efficiency by tracking only the current path and best f-value, unlike A*, which stores comprehensive open and closed lists.
- **Performance & Behavior:**
  - **A*:** Guarantees optimal pathfinding with an admissible heuristic.
  - **RBFS:** Not strictly guaranteed to be optimal due to its recursive backtracking approach.

Summary Conclusion:

A* outperforms RBFS in time efficiency and optimality when sufficient memory is available. Conversely, RBFS is ideal for memory-constrained environments, though it may incur redundant path exploration. Detailed comparative analysis can be found in.

3 PRACTICAL - 3: Decision Tree Learning

Aim:

1. Implement the Decision Tree Learning algorithm to build a decision tree for a given dataset.
2. Evaluate the accuracy and effectiveness of the decision tree on test data.
3. Visualize and interpret the generated decision tree.

Theory:

A **Decision Tree** is a supervised machine learning algorithm used for both classification and regression tasks. It's a tree-like model that makes decisions based on a series of conditions or rules learned from the training data.

Each internal node in the tree represents a decision rule based on a feature, and each leaf node represents a class label (in classification) or a predicted value (in regression). Decision trees are characterized by their simplicity, interpretability, and ability to handle both categorical and numerical data.

Data Set:

We download the Iris data set and use it for the present case. As we are performing the practical on Google Colab, we upload the dataset as `iris.csv`.

Python Code:

```
# Decision tree
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load your dataset from a local file (e.g., CSV)
# Replace 'your_dataset.csv' with the actual path to your dataset file
# Download the required dataset from kaggle
data = pd.read_csv('iris.csv')
print(data.head())

# Assuming the target variable is in a column named 'variety'
X = data.drop('variety', axis=1)
y = data['variety']

# Split the dataset into a training set and a testing set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create a Decision Tree classifier
clf = DecisionTreeClassifier()

# Fit the classifier to the training data
clf.fit(X_train, y_train)

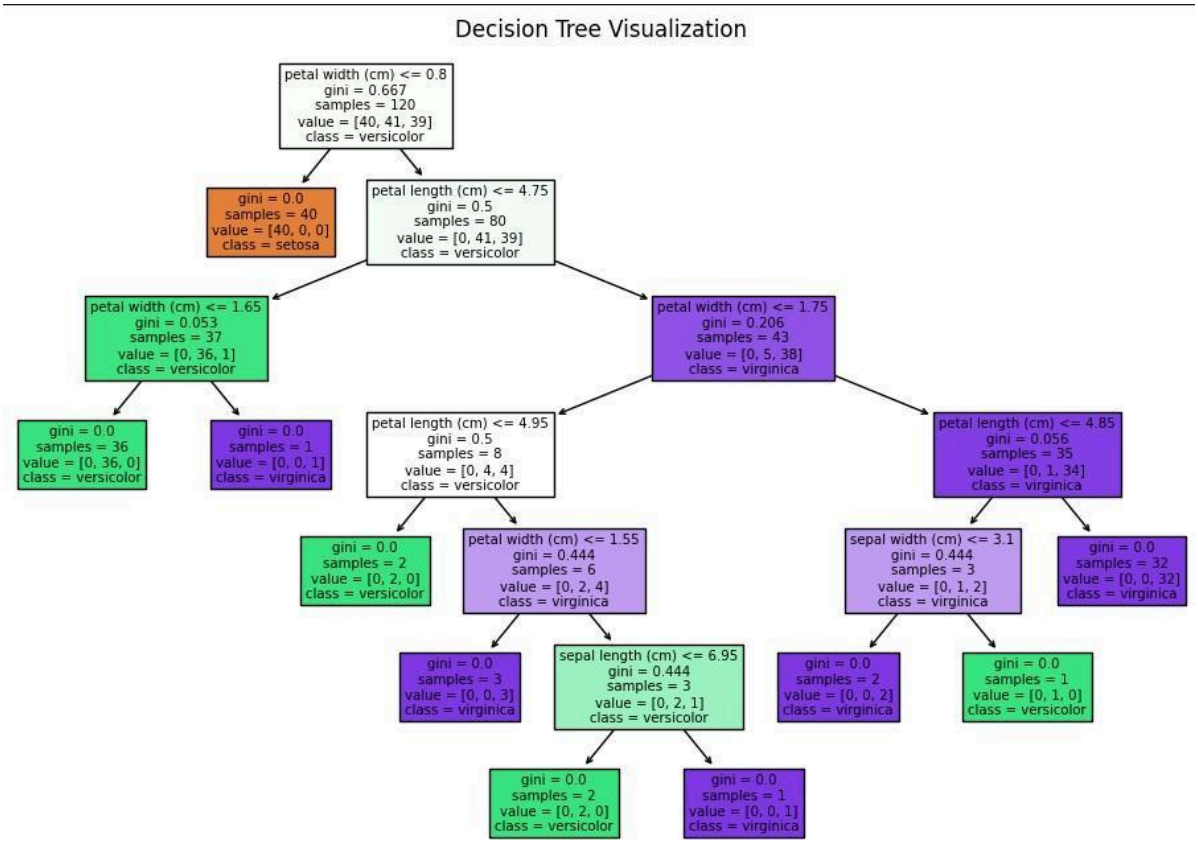
# Make predictions on the test data
y_pred = clf.predict(X_test)

# Calculate the accuracy of the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")

# Visualize and interpret the generated decision tree
plt.figure(figsize=(12, 8))
plot_tree(clf, filled=True, feature_names=X.columns, class_names=y.unique().astype(str))
plt.title("Decision Tree Visualization")
plt.show()
```

Output:

```
Accuracy: 1.00
```



4 PRACTICAL - 4: Feedforward Backpropagation NeuralNetwork

Aim:

1. Implement the Feed Forward Backpropagation algorithm to train a neural network.
2. Use a given dataset to train the neural network for a specific task.
3. Evaluate the performance of the trained network on test data.

Theory:

Feedforward neural networks, also known as feedforward neural networks (FNNs) or multilayer perceptrons (MLPs), are a fundamental type of artificial any neural network used in machine learning and deep learning. They are designed to model complex relationships between inputs and outputs by stacking multiple layers of interconnected neurons (also called nodes or units). Feedforward neural networks are a foundational component of deep learning, allowing machines to learn and make predictions from data by forming increasingly abstract and hierarchical representations through multiple layers of interconnected neurons.

For the present case we implement the XOR-operation using Feedforward Backpropagation NeuralNetwork. The XOR-operation for a 2-input variable is as follows:

Inputs (X_1)	Inputs (X_2)	Target (T)
0	0	0
0	1	1
1	0	1
1	1	0

Python Code:

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)
```

```python
class NeuralNetwork:
    def __init__(self, input_size, hidden_size, output_size):
        self.weights_input_hidden = np.random.uniform(-1, 1, (input_size, hidden_size))
        self.weights_hidden_output = np.random.uniform(-1, 1, (hidden_size, output_size))

    def forward(self, inputs):
        self.hidden_output = sigmoid(np.dot(inputs, self.weights_input_hidden))
        self.predicted_output = sigmoid(np.dot(self.hidden_output, self.weights_hidden_output))
        return self.predicted_output

    def backward(self, inputs, target, learning_rate):
        error = target - self.predicted_output
        delta_output = error * sigmoid_derivative(self.predicted_output)
        error_hidden = delta_output.dot(self.weights_hidden_output.T)
        delta_hidden = error_hidden * sigmoid_derivative(self.hidden_output)

        self.weights_hidden_output += np.outer(self.hidden_output, delta_output) * learning_rate
        self.weights_input_hidden += np.outer(inputs, delta_hidden) * learning_rate

    def train(self, training_data, targets, epochs, learning_rate):
        for epoch in range(epochs):
            for i in range(len(training_data)):
                inputs = training_data[i]
                target = targets[i]
                self.forward(inputs)
                self.backward(inputs, target, learning_rate)

    def predict(self, inputs):
        return self.forward(inputs)

# XOR dataset
training_data = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
targets = np.array([[0], [1], [1], [0]])

input_size, hidden_size, output_size = 2, 4, 1
learning_rate, epochs = 0.1, 10000

nn = NeuralNetwork(input_size, hidden_size, output_size)
nn.train(training_data, targets, epochs, learning_rate)

# Test the trained network
for i in range(len(training_data)):
    inputs = training_data[i]
    prediction = nn.predict(inputs)
    print(f"Input: {inputs}, Predicted Output: {prediction}")
```

Output:

```
Input: [0 0], Predicted Output: [0.0970792]
Input: [0 1], Predicted Output: [0.92842484]
Input: [1 0], Predicted Output: [0.92293184]
Input: [1 1], Predicted Output: [0.05847459]
```

5 PRACTICAL - 5: Support Vector Machine

Aim:

1. Implement the SVM algorithm for binary classification.
2. Train an SVM model using a given dataset and optimize its parameters.
3. Evaluate the performance of the SVM model on test data and analyze the results.

Theory:

Support Vector Machines (SVM) are a powerful and versatile class of supervised machine learning algorithms used for classification and regression tasks.

Data Set:

We download the Iris data set and use it for the present case. As we are performing the practical on Google Colab, we upload the dataset as `iris.csv`.
(Note: The script reads from `diabetes.csv` to predict the `Age` column).

Python Code:

```python
# Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
```

```python
from sklearn.metrics import accuracy_score

# Load your dataset from a local file (e.g., CSV)
# Replace 'your_dataset.csv' with the actual path to your dataset file
data = pd.read_csv('diabetes.csv')

# Assuming the target variable is in a column named 'Age'
X = data.drop('Age', axis=1)
y = data['Age']

# Split the dataset into a training set and a testing set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create an SVM classifier
svm_classifier = SVC(kernel='linear')
# You can choose different kernels here

# Fit the classifier to the training data
svm_classifier.fit(X_train, y_train)

# Make predictions on the test data
y_pred = svm_classifier.predict(X_test)

# Calculate the accuracy of the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
```

Output:

```
Accuracy: 0.08
```

6 PRACTICAL - 6: Naïve Bayes Classifier

Aim:

1. To implement the Naïve Bayes' algorithm for classification.
2. Train a Naïve Bayes' model using a given dataset and calculate class probabilities.
3. Evaluate the accuracy of the model on test data and analyze the results.

Theory:

Naïve Bayes is a family of probabilistic machine learning algorithms based on Bayes' Theorem. It's particularly useful for classification tasks, where you want to predict the class label of an input based on its features. Despite its "Naïve" assumption, Naïve Bayes has been shown to work surprisingly well in many real-world scenarios, especially in text classification and other high-dimensional datasets.

1. Bayes' Theorem:

Naïve Bayes is built upon Bayes' Theorem, which is a fundamental concept in probability theory. Bayes' Theorem describes how to update the probability of a hypothesis (an event) based on new evidence. It's written as:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Where:

- $P(A|B)$ is the probability of event A given event B.
- $P(B|A)$ is the probability of event B given event A.
- $P(A)$ is the prior probability of event A.
- $P(B)$ is the evidence or marginal probability of event B.

2. Naïve Assumption:

The "Naïve" in Naïve Bayes refers to the assumption that the features (attributes) used to predict the class are conditionally independent of each other given the class label. In other words, the presence or absence of a particular feature doesn't affect the presence or absence of any other feature. This simplifies the calculations significantly and makes the algorithm computationally efficient.

3. Training:

During the training phase, Naïve Bayes estimates the probabilities needed for classification. It calculates the prior probabilities of each class based on the training data and then calculates the likelihood of each feature given each class.

Example: If you're classifying emails as "spam" or "not spam," you'd estimate the probability of certain words appearing in spam emails versus non-spam emails.

4. Prediction:

When making a prediction for a new input, Naïve Bayes calculates the probability of the input belonging to each class based on the features. It uses Bayes' Theorem to compute the posterior probability for each class, and the class with the highest posterior probability is chosen as the predicted class.

Types of Naïve Bayes Classifiers:

There are several variants of Naïve Bayes classifiers, including:

- **Gaussian Naïve Bayes:** Assumes that the features follow a Gaussian distribution.
- **Multinomial Naïve Bayes:** Used for discrete data, often in text classification (e.g., counting word occurrences).
- **Bernoulli Naïve Bayes:** Used for binary data, such as presence or absence of features.

Python Code:

(Note: To run this code, Scikit-Learn must be installed: `pip install scikit-learn`*)*

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create a Naïve Bayes classifier (Gaussian Naïve Bayes for continuous features)
clf = GaussianNB()

# Train the classifier on the training data
clf.fit(X_train, y_train)

# Make predictions on the test data
y_pred = clf.predict(X_test)

# Calculate and print the accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

Output:

```
Accuracy: 1.0
```

7 PRACTICAL - 7: K - Nearest Neighbors (K-NN)

Aim:

1. Implement the K-NN algorithm for classification or regression.
2. Apply the K-NN algorithm to a given dataset and predict the class or value for test data.
3. Evaluate the accuracy or error of the predictions and analyze the results.

Theory:

K-NN stands for "K-Nearest Neighbors," and it is a simple and widely used machine learning algorithm for classification and regression tasks.

K-NN is an instance-based learning method, which means it makes predictions based on the similarity of new data points to existing data points in the training dataset. Here's how K-NN works:

- **Training:** The algorithm stores the entire training dataset in memory, including both the feature values (attributes) and their corresponding class labels (for classification) or target values (for regression).
- **Prediction:** When making a prediction for a new, unseen data point, K-NN finds the K nearest data points (neighbors) to the new point from the training dataset. "K" is a user-defined parameter representing the number of neighbors to consider. These neighbors are found based on a distance metric, typically Euclidean distance, but other distance metrics are also possible.
- **Classification:** In a classification task, K-NN takes a majority vote among the K nearest neighbors to determine the class label of the new data point. The class label that occurs most frequently among the K neighbors is assigned to the new point.
- **Regression:** In a regression task, K-NN calculates the average (or weighted average) of the target values of the K nearest neighbors and assigns this average as the predicted target value for the new data point.

Python Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load the dataset from the CSV file
df = pd.read_csv('Iris_csv.csv')

# Extract relevant columns for the feature matrix (exclude 'ID' and 'Target' columns)
X = df.drop(['ID', 'Target'], axis=1).values

# Target variable for classification
y = df['Target'].values

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Define the value of k for K-NN
k = 3

# Classification with K-NN
clf = KNeighborsClassifier(n_neighbors=k)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

# Evaluate classification accuracy
classification_accuracy = accuracy_score(y_test, y_pred)
print("Classification Accuracy:", classification_accuracy)
```

Output:

```
Classification Accuracy: 1.0
```

8 PRACTICAL - 8: Demo of TensorFlow Tools

Aim:

1. Explore and experiment with TensorFlow tools and libraries.
2. Perform a demonstration or mini-project showcasing the capabilities of the tools.
3. Discuss and present the findings and potential applications.

Theory:

Introduction to TensorFlow:

TensorFlow is an open-source machine learning framework developed by the Google Brain team. It is designed to facilitate the development of machine learning and deep learning models, particularly neural networks. TensorFlow is widely used in various fields, including computer vision, natural language processing, speech recognition, and more.

Key aspects about TensorFlow:

1. **Tensors:** TensorFlow is named after its core data structure, the “tensor.” A tensor is a multi-dimensional array, which can be thought of as a generalization of vectors and matrices to higher dimensions. Tensors are fundamental for representing and manipulating data in machine learning models.
2. **Graph-based computation:** TensorFlow uses a graph-based approach to define and execute machine learning models. In this approach, you first create a computational graph that represents the operations in your model. Then, you feed data into this graph to perform computations. This allows for efficient execution, optimization, and distribution of computations across CPUs and GPUs.
3. **Flexibility:** TensorFlow offers flexibility in designing and implementing machine learning models. You can build various types of models, including feedforward neural networks, convolutional neural networks (CNNs), recurrent neural networks (RNNs), and more. TensorFlow also supports custom model architectures and operations, making it suitable for research and production applications.
4. **High-level APIs:** TensorFlow provides high-level APIs like Keras, which is integrated into TensorFlow as its official high-level API. Keras simplifies the process of building and training deep learning models, making it accessible to developers with varying levels of experience.
5. **Ecosystem:** TensorFlow has a rich ecosystem of libraries and tools that extend its functionality. Some of these include TensorFlow Serving for deploying models in production, TensorFlow Lite for mobile and embedded devices, and TensorFlow.js for running models in web browsers.

We will have a practical demonstration using TensorFlow and a dataset for spam email classification.

Before running the code, we create two files containing sample emails, one is spam mail and the other is non-spam email, and save them as `Spam.txt` and `NSpam.txt`.

Spam.txt

```
Subject: Congratulations! You've Won $100,000 Cash Prize
Dear George, I am thrilled to inform you that you are the lucky winner of our recent contest and have been
awarded a cash prize of $100,000! Your participation and enthusiasm are truly appreciated, and we couldn't
be happier to share this exciting news with you. Regards idontSmile
```

NSpam.txt

```
Subject: Invitation for Dinner
Dear Friend, I hope this email finds you well. I wanted to extend a warm invitation to you for a
dinner party on 18/8/2023 at my home next Friday, and it would be wonderful to have you join us.
Best regards, iSmile
```

Python Code:

```python
import tensorflow as tf
import numpy as np
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Step 1: Prepare a dataset of labeled emails (spam and non-spam)
emails = [
    "Buy cheap watches! Free shipping!",
    "Meeting for lunch today?",
    "Claim your prize! You've won $1,000,000!",
    "Important meeting at 3 PM.",
]
labels = [1, 0, 1, 0]

# Step 2: Tokenize and pad the email text data
max_words = 1000
max_len = 50
tokenizer = Tokenizer(num_words=max_words, oov_token="<OOV>")
tokenizer.fit_on_texts(emails)
sequences = tokenizer.texts_to_sequences(emails)
X_padded = pad_sequences(sequences, maxlen=max_len, padding="post", truncating="post")

# Step 3: Define the neural network model
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(input_dim=max_words, output_dim=16, input_length=max_len),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(16, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Step 4: Define training data and labels as NumPy arrays
training_data = np.array(X_padded)
training_labels = np.array(labels)

# Step 5: Train the model
model.fit(training_data, training_labels, epochs=10)

# Step 6: Test if 'Spam.txt' or 'NSpam.txt' is spam or not
file_path = "Spam.txt"  # Or switch to "NSpam.txt" to run the respective test
with open(file_path, "r", encoding="utf-8") as file:
    sample_email_text = file.read()

# Tokenize and pad the sample email text
sequences_sample = tokenizer.texts_to_sequences([sample_email_text])
sample_email_padded = pad_sequences(sequences_sample, maxlen=max_len, padding="post", truncating="post")

# Use the trained model to make predictions
prediction = model.predict(sample_email_padded)

# Set a classification threshold (e.g., 0.5)
threshold = 0.5

# Classify the sample email based on the threshold
if prediction > threshold:
    print(f"Sample Email ('{file_path}'): SPAM")
else:
    print(f"Sample Email ('{file_path}'): NOT SPAM")
```

Output (Testing NSpam.txt):

```
Epoch 1/10
1/1 [==============================] - 1s 963ms/step - loss: 0.6952 - accuracy: 0.5000
Epoch 2/10
1/1 [==============================] - 0s 13ms/step - loss: 0.6904 - accuracy: 0.7500
Epoch 3/10
1/1 [==============================] - 0s 14ms/step - loss: 0.6854 - accuracy: 0.7500
```

```
Epoch 4/10
1/1 [==============================] - 0s 15ms/step - loss: 0.6816 - accuracy: 1.0000
Epoch 5/10
1/1 [==============================] - 0s 28ms/step - loss: 0.6782 - accuracy: 1.0000
Epoch 6/10
1/1 [==============================] - 0s 12ms/step - loss: 0.6745 - accuracy: 1.0000
Epoch 7/10
1/1 [==============================] - 0s 17ms/step - loss: 0.6708 - accuracy: 1.0000
Epoch 8/10
1/1 [==============================] - 0s 28ms/step - loss: 0.6669 - accuracy: 1.0000
Epoch 9/10
1/1 [==============================] - 0s 24ms/step - loss: 0.6630 - accuracy: 1.0000
Epoch 10/10
1/1 [==============================] - 0s 13ms/step - loss: 0.6589 - accuracy: 1.0000
1/1 [==============================] - 0s 139ms/step
Sample Email ('NSpam.txt'): NOT SPAM
```

Output (Testing Spam.txt):

```
Epoch 1/10
1/1 [==============================] - 1s 934ms/step - loss: 0.6922 - accuracy: 0.7500
Epoch 2/10
1/1 [==============================] - 0s 13ms/step - loss: 0.6877 - accuracy: 0.5000
Epoch 3/10
1/1 [==============================] - 0s 11ms/step - loss: 0.6822 - accuracy: 1.0000
Epoch 4/10
1/1 [==============================] - 0s 14ms/step - loss: 0.6772 - accuracy: 1.0000
Epoch 5/10
1/1 [==============================] - 0s 14ms/step - loss: 0.6723 - accuracy: 1.0000
Epoch 6/10
1/1 [==============================] - 0s 14ms/step - loss: 0.6677 - accuracy: 1.0000
Epoch 7/10
1/1 [==============================] - 0s 14ms/step - loss: 0.6622 - accuracy: 1.0000
Epoch 8/10
1/1 [==============================] - 0s 16ms/step - loss: 0.6567 - accuracy: 1.0000
Epoch 9/10
1/1 [==============================] - 0s 11ms/step - loss: 0.6512 - accuracy: 1.0000
Epoch 10/10
1/1 [==============================] - 0s 12ms/step - loss: 0.6456 - accuracy: 1.0000
1/1 [==============================] - 0s 68ms/step
Sample Email ('Spam.txt'): SPAM
```

9 PRACTICAL - 9: Adaboost Learning Algorithm

AdaBoost (Adaptive Boosting), introduced by Yoav Freund and Robert Schapire in 1996, is an iterative ensemble method that combines multiple weak classifiers to build a high-accuracy strong classifier. It adjusts data sample weights in each iteration to improve predictions on difficult observations.

Building Model in Python

```python
# Load libraries
from sklearn.ensemble import AdaBoostClassifier
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn import metrics

# Load data
iris = datasets.load_iris()
X = iris.data
y = iris.target

# Split dataset into training set and test set (70% training and 30% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

# Create and train Adaboost classifier object
abc = AdaBoostClassifier(n_estimators=50, learning_rate=1)
model = abc.fit(X_train, y_train)

# Predict and calculate accuracy
y_pred = model.predict(X_test)
print("Accuracy:", metrics.accuracy_score(y_test, y_pred))
```

Output:

```
Accuracy: 0.9333333336
```

10 PRACTICAL - 10: Association Rule Mining

Association rule mining is a technique used to identify patterns in large datasets. It involves finding relationships between variables in the data and using those relationships to make predictions or decisions. The goal of association rule mining is to uncover rules that describe the relationships between different items in the dataset.

Example: Consider a dataset of transactions at a grocery store. Association rule mining could be used to identify relationships between items that are frequently purchased together. For example, the rule "If a customer buys bread, they are also likely to buy milk" is an association rule that could be mined from this dataset. We can use such rules to inform decisions about store layout, product placement, and marketing efforts.

Association rule mining typically involves using algorithms to analyze the data and identify the relationships. These algorithms can be based on statistical methods or machine learning techniques. The resulting rules are often expressed in the form of "if-then" statements, where the "if" part represents the antecedent (the condition being tested) and the "then" part represents the consequent (the outcome that occurs if the condition is met).

Python Code:

```python
# Import necessary libraries
from mlxtend.frequent_patterns import apriori
from mlxtend.frequent_patterns import association_rules
import pandas as pd

# Create a sample dataset
data = {
    'TID':,
    'Items': [['A', 'B', 'C'], ['Myself', 'B', 'D'], ['Maity', 'A'], ['Myself', 'B'], ['Myself', 'C']]
}
df = pd.DataFrame(data)

# Convert the dataset into a one-hot encoded format
oht = df['Items'].str.join('|').str.get_dummies()
oht = oht.set_index(df['TID'])

# Find frequent itemsets using the Apriori algorithm
frequent_itemsets = apriori(oht, min_support=0.6, use_colnames=True)

# Find association rules
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1.0)

# Display the frequent itemsets and association rules
print("Frequent Itemsets:")
print(frequent_itemsets)
print("\nAssociation Rules:")
print(rules)
```

Output:

```
Frequent Itemsets:
   support itemsets
0      0.8      (A)
1      0.8      (B)
2      0.6      (C)
3      0.6   (A, B)

Association Rules:
Empty DataFrame
Columns: [antecedents, consequents, antecedent support, consequent support, support, confidence,
         lift, leverage, conviction, zhangs_metric]
Index: []
```